CaMDN: Enhancing Cache Efficiency for Multi-tenant DNNs on Integrated NPUs

Tianhao Cai[1,2], Liang Wang[1,2,*], Limin Xiao[1,2,*], Meng Han[3], Zeyu Wang[1,2], Lin Sun[4] and Xiaojian Liao[1,2]
[1] State Key Laboratory of CCSE, Beihang University, Beijing, China
[2] School of Computer Science and Engineering, Beihang University, Beijing, China
[3] School of Integrated Circuits, Tsinghua University, Beijing, China
[4] Jiangsu Shuguang Optoelectric Co., Ltd., Yangzhou, China
caitianhao@buaa.edu.cn, lwang20@buaa.edu.cn, xiaolm@buaa.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132424

***Abstract*—With the rapid development of DNN applications, multi-tenant execution, where multiple DNNs are co-located on a single SoC, is becoming a prevailing trend. Although many methods are proposed in prior works to improve multi-tenant performance, the impact of shared cache is not well studied. This paper proposes CaMDN, an architecture-scheduling co-design to enhance cache efficiency for multi-tenant DNNs on integrated NPUs. Specifically, a lightweight architecture is proposed to support model-exclusive, NPU-controlled regions inside shared cache to eliminate unexpected cache contention. Moreover, a cache scheduling method is proposed to improve shared cache utilization. In particular, it includes a cache-aware mapping method for adaptability to the varying available cache capacity and a dynamic allocation algorithm to adjust the usage among co-located DNNs at runtime. Compared to prior works, CaMDN reduces the memory access by 33.4% on average and achieves a model speedup of up to 2.56× (1.88× on average).**

***Index Terms*—Cache, Multi-tenant, DNN, NPU, SoC.**

I. Introduction

Deep Neural Networks (DNNs) are increasingly pervasive in various applications. This brings an inevitable demand for efficient multi-tenant DNN execution, where many models are co-located on a single SoC to improve the performance of comprehensive DNN applications. Therefore, modern SoCs tend to integrate Neural Processing Units (NPUs) alongside CPUs to support trending multi-DNN applications [1]–[3].

The key challenge of co-locating DNNs is the intense contention among running tasks over all kinds of shared resources. Methods are proposed to schedule different resources among models through either temporal interleaving [4]–[6] or spatial partitioning [7]–[14]. However, these methods mainly focus on memory bandwidth [7], [8], NPU cores [9], [10], or both [11]–[13]. The performance impact of shared cache with multi-tenant DNNs is still not well studied.

Cache efficiency can be severely compromised in multi-tenancy due to the contention among running programs. Our experiment shows that cache hit rate drops by up to 59.7%, and memory access increases by up to 64.1% with 32 co-located DNNs, as shown in Section II-C. Since multi-tenant DNNs are generally memory-bound, the increase in memory access directly results in severe model slowdown. According to our statistical analyses, a large portion of data in multi-tenant DNNs is either non-reusable or reused over long distances on shared cache, which is a primary cause of the inefficiency.

Prior works manage to improve cache efficiency with different types of multi-tenant workloads other than DNNs, such as CPU workloads [15]–[18] and accelerator workloads [19]–[21]. However, the performance improvements brought by these methods are limited due to either the need to keep cache transparent to workloads, or lack of consideration on special characteristics of trending multi-tenant DNNs.

For multi-tenant DNNs, both architecture and scheduling require specialized optimization to improve shared cache efficiency. In terms of architecture, cache is required to be able to bypass non-reusable data and retain data with long reuse distances. In terms of scheduling, model mapping is required to be aware that cache will be shared among multiple models, and resource allocation is required to dynamically respond to changes of cache usage during multi-tenant execution.

Therefore, this paper proposes CaMDN, an architecture-scheduling co-design to enhance cache efficiency for multi-tenant DNNs on integrated NPUs. Specifically, CaMDN introduces a lightweight architecture to support model-exclusive, NPU-controlled regions inside shared cache. This allows NPUs to manage data storage and replacement on cache and eliminates unexpected contention among models. To fully utilize the architectural advantages, CaMDN provides a cache scheduling method including cache-aware mapping and dynamic cache allocation. In particular, cache-aware mapping generates multiple mapping candidates that target different cache usage levels to provide adaptability to the varying available capacity. Dynamic cache allocation adjusts cache usage among co-located DNNs at runtime. It responds quickly to changes of cache usage to improve cache utilization. In summary, this paper makes the following main contributions:

- We propose a lightweight architecture to support model-exclusive, NPU-controlled regions inside shared cache.
- We propose a cache scheduling method that contains cache-aware DNN mapping and dynamic cache allocation to improve shared cache utilization.
- We demonstrate that, compared to prior works, CaMDN reduces the memory access by 33.4% and achieves a model speedup of up to 2.56× (1.88× on average).

This work was supported by the National Key R&D Program of China under Grant No. 2023YFB4503100, the National Natural Science Foundation of China under Grant No. 62272026 and No.62104014, the Fundamental Research Funds for the Central Universities, State Key Laboratory of Complex & Critical Software Environment under Grant No. CCSE-2024ZX-10, and Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing.

* Liang Wang and Limin Xiao are corresponding authors of this paper.

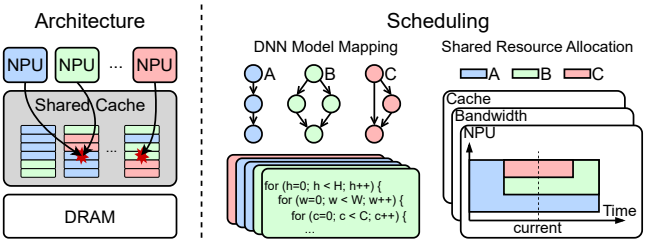

Fig. 1: An overview of architecture and scheduling for multi-tenant DNN execution on integrated NPUs. Cache contention occurs between co-located DNNs.

II. Background and Motivation

A. Multi-tenant DNN Execution

As DNNs increasingly pervade various applications, multi-tenant execution becomes an inevitable requirement for multiple reasons. First, complex applications such as AR/VR are likely to apply more than one DNN. Second, multiple DNN applications may run on a single device like personal computers and edge devices [22]. Third, cloud servers need to support multi-tenancy for modern AI services [23].

Figure 1 illustrates an overview of architecture and scheduling for multi-tenant DNN execution on a single SoC. Architecturally, to meet the trending demand for multi-tenant DNNs, the chip industry has started to integrate NPUs alongside CPUs on their SoCs [1]–[3]. Such architectures leverage the benefits of shared memory subsystem, including higher interconnect bandwidth, reduced host-device data movement, and additional on-chip storage provided by shared cache.

Scheduling refers to the process of deploying DNN models on target hardware. In particular, model mapping is to generate hardware-specific programs from high-level model descriptions. Mapping is demonstrated to have a significant impact on model performance, and many optimizations are proposed in prior works [24]–[26]. At runtime, shared resources, such as NPUs and bandwidth, are allocated to models according to their requirements as well as target deadlines.

B. Scheduling Multi-tenant DNNs

Recently, methods are proposed in prior works to schedule multi-tenant DNNs on integrated NPUs. Some of them use temporal interleaving to execute tasks in turn [4]–[6]. These methods suffer from under-utilization as resources scale richer. Others use spatial partitioning to fully utilize shared resources. We categorize these methods according to the resources they mainly focus on.

1) Bandwidth: MAGMA [7] and MoCA [8] allocate memory bandwidth usage among co-located DNNs according to their memory access requirements.

2) Compute: Planaria [9] manages to co-locate multiple models on a single large systolic array-based NPU. DREAM [10] dynamically schedules multiple NPUs for multi-tenant DNNs in real-time scenarios.

3) Bandwidth & Compute: RELMAS [11] considers bandwidth contention for NPU scheduling but does not employ any hardware constraints on bandwidth usage. HDA [12] and AuRORA [13] leverage static and dynamic scheduling methods, respectively, to co-allocate bandwidth and NPUs.

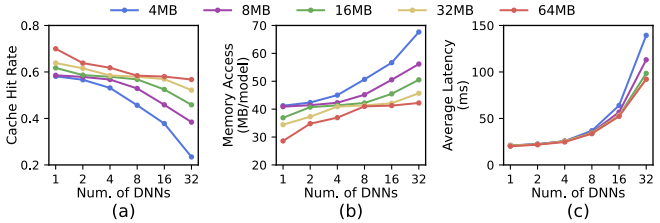

Fig. 2: Cache hit rate, memory access and average latency with different settings on number of DNNs and cache capacity.

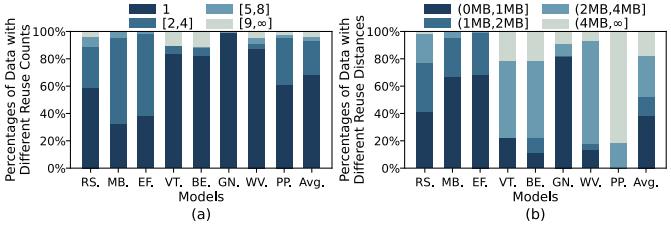

Fig. 3: Percentages of data with different reuse counts and reuse distances on shared cache in benchmark DNN models.

Prior works above manage to schedule memory bandwidth and/or NPUs to achieve higher performance. However, they fail to consider the performance impact of shared cache with multi-tenant DNNs. Veltair [14] uses cache performance counters for contention monitoring, but does not utilize any hardware method to address the issue. Additionally, Veltair targets server-level many-core CPUs rather than NPUs.

C. Cache Inefficiency with Multi-tenant DNNs

To understand the performance impact of shared cache with multi-tenant DNNs, we conduct an experiment that randomly dispatches different DNN models on an NPU-integrated SoC. The experiment is performed with different settings on the number of co-located models and shared cache capacity. The detailed experimental setup is described in Section IV-A. Figure 2 shows the results of the experiment. The cache hit rate drops by 18.9% to 59.7%, and the memory access increases by 32.7% to 64.1% as the number of DNNs reaches 32. This is due to the severe cache contention among models, which occurs when a process frequently replaces cache lines belonging to other co-running processes, as illustrated in the left part of Figure 1. As a result, the average model latency increases by 3.46× to 5.65×. Since multi-tenant DNNs are generally memory-bound, the increase in memory access directly contributes to the slowdown.

To further understand how DNN workload characteristics incur the inefficiency of shared cache, we perform the following statistical analyses on benchmark DNNs in Table I. Figure 3(a) shows the percentages of data with different reuse counts. A reuse count refers to the expected number of repeated cache accesses to a piece of data. On average, 68.0% of data have no future reuse but still occupy cache space, which results in a large amount of reusable data being early replaced off-chip. Figure 3(b) shows the percentages of intermediate data with different reuse distances when accessed across DNN layers. A reuse distance refers to the amount of accesses to other data that occur between two repeated accesses to the same piece of data. On average, 61.8% of intermediate data have reuse

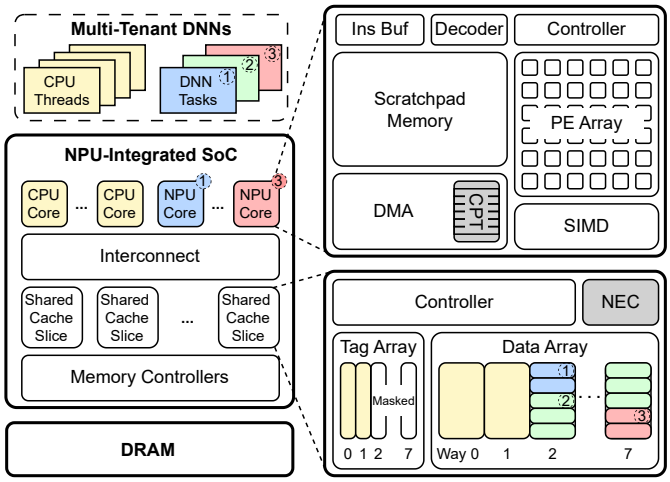

Fig. 4: The overview of CaMDN architecture. CaMDN installs an NEC in each shared cache slice and a CPT in each NPU.

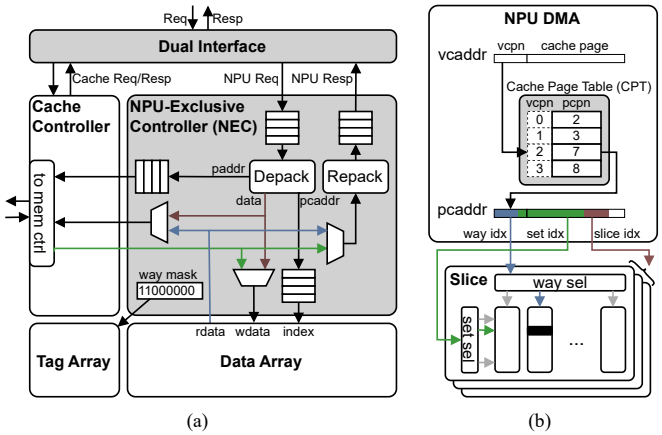

Fig. 5: Implementation details of CaMDN architecture. (a) NEC handles NPU-specific requests. (b) CPT translates *vcaddr* into *pcaddr* that indexes the targeted cache line in data arrays.

distances more than 1MB, and 47.9% of them more than 2MB. Such large reuse distances make it difficult for transparent cache to retain these data for future reuse, especially when multiple models are competing for cache space. In summary, a large portion of data in DNNs is either non-reusable or reused over long distances. Shared cache shows evident inefficiency to handle such data, therefore significantly degrading the performance of multi-tenant DNN execution.

D. Enhancing Cache Efficiency

Prior works manage to enhance cache efficiency for different multi-tenant workloads other than DNNs. For CPU workloads, cache partitioning and related techniques are proposed to limit the cache usage of each process [15]–[18]. However, these methods offer limited performance improvement, since cache must be hardware-managed and transparent to CPU workloads. For accelerator workloads, methods are purposed that either instantiate accelerator-controlled buffers inside shared cache [19], [20], or dynamically change cache coherence mode for each workload at runtime [21]. However, these methods lack consideration for the special characteristics of multi-tenant DNNs.

For multi-tenant DNNs, both architecture and scheduling require specialized optimization to improve shared cache efficiency. In terms of architecture, cache is required to be able to bypass non-reusable data and retain data with long reuse distances. In terms of scheduling, mapping is required to be aware that cache will be shared among multiple models, and resource allocation is required to dynamically respond to changes of cache usage during multi-tenant execution. Therefore, this paper proposes CaMDN, an architecture-scheduling co-design to enhance cache efficiency for multi-tenant DNNs on integrated NPUs. Specifically, CaMDN contains a lightweight architecture to support model-exclusive, NPU-controlled regions inside shared cache, a cache-aware DNN mapping method for adaptability to the varying available cache capacity, and a dynamic cache allocation algorithm to adjust the cache usage among co-located DNNs at runtime.

III. CaMDN System

A. Overview

The architecture-scheduling co-design of CaMDN contains the following components. (1) An NPU-controlled cache architecture that supports model-exclusive, NPU-controlled regions inside shared cache by adding an NPU-exclusive controller (NEC) in each cache slice and a cache page table (CPT) in each NPU (Section III-B). (2) A cache-aware mapping method that provides adaptability to varying available cache size by generating multiple mapping candidates that target different cache usage levels (Section III-C). (3) A dynamic cache allocation algorithm that adjusts cache usage among models by predicting near-future cache usage and selecting mapping candidates accordingly (Section III-D).

B. NPU-controlled Cache Architecture

The NPU-controlled cache architecture of CaMDN supports the instantiation of model-exclusive, NPU-controlled regions inside shared cache. Specifically, (1) shared cache is divided into a general-purpose subspace and an NPU subspace by way partitioning. (2) An NEC is added in each cache slice to take control of the NPU subspace and provide NPU-controlled cache access semantics. (3) The NPU subspace is then divided into model-exclusive regions by paging, which is implemented by a hardware-based CPT inside each NPU.

1) Way-Partitioned NPU Subspace: CaMDN divides shared cache into a general-purpose subspace and an NPU subspace by way partitioning. This eliminates unexpected contention between CPU applications and NPU tasks. CaMDN implements way partitioning by adding a way mask register in each cache slice to mask off the ways allocated to the NPU subspace. As shown in the bottom-right part of Figure 4, ways 0-1 are reserved for CPU applications, and ways 2-7 are assigned to the NPU subspace. Different proportions of partitioning can be adapted for different application scenarios.

2) NPU-controlled Access: To support NPU-controlled cache access, CaMDN adds a lightweight NEC and a dual interface in each cache slice to take control of the NPU subspace, as shown in Figure 5(a). The dual interface provides

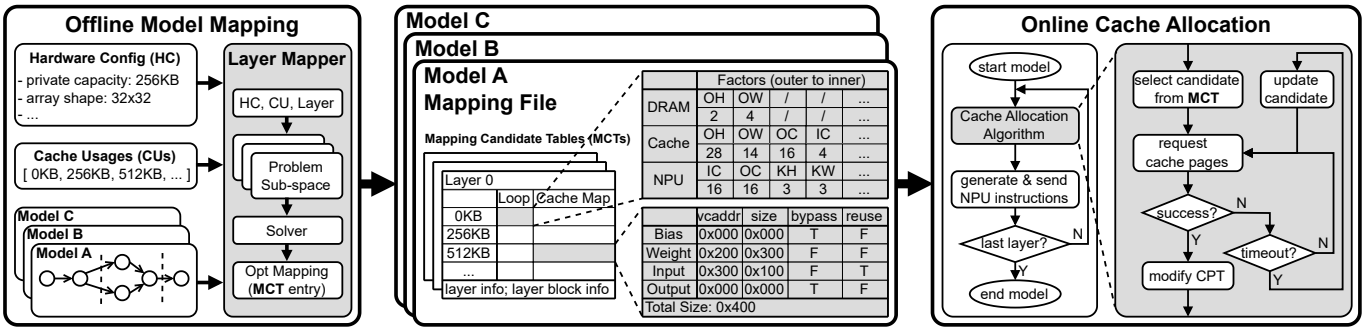

Fig. 6: CaMDN cache scheduling includes offline cache-aware mapping and online cache allocation.

a uniform interface for normal cache requests and NPU-specific requests. NEC handles different types of NPU requests and performs operations including reading/writing cache lines and generating memory requests to memory controllers.

NEC implements NPU-controlled cache access semantics with cache line granularity. Replacing the original hardware-managed cache control, these semantics provide NPU programs with full control over data movement, storage and replacement. In addition to basic semantics which implement memory-cache and cache-NPU data movements, NEC introduces advanced semantics for performance improvement: bypass and multicast. Bypass semantics support bypassing non-reusable data around shared cache, reserving cache space for reusable data to improve utilization. This includes (1) **bypass-read** a line from memory to NPU; (2) **bypass-write** a line from NPU to memory. Multicast semantics help reduce bandwidth pressure on both memory and NoC by combining identical requests from a group of NPUs that are allocated to the same model. This includes (3) **multicast-read** a line from cache to a group of NPUs; (4) **multicast-bypass-read** a line from memory to a group of NPUs.

3) Virtual Cache Addressing: To further make the NPU subspace divisible and addressable for co-located models, CaMDN utilizes hardware-based paging to create isolated model-exclusive regions and provide an independent virtual address space for every model. As shown in the bottom-right part of Figure 4, the NPU subspace is divided into pages of identical size and assigned to different models. A hardware-based CPT is installed in each NPU to translate virtual cache address (*vcaddr*) into physical cache address (*pcaddr*). Considering that NPUs usually operate on data chunks of at least dozens of KB, we set the page size to 32KB for a 16MB shared cache. Thus, a hardware-based CPT has at most 512 entries, each of which needs at most 3 bytes to store a physical cache page number (*pcpn*) and a valid bit, resulting in a total 1.5KB SRAM overhead, which is negligible as demonstrated in Section IV-B4.

Figure 5(b) illustrates the address translation and cache line indexing. The virtual cache page number (*vcpn*) of a *vcaddr* is used to index the CPT for *pcpn*, which is then used to compose *pcaddr*. *pcaddr* is divided into four bit-fields: byte offset, slice index, set index and way index, from lower to higher, which uniquely determine the targeted cache line. In this indexing manner, consecutive data lines are distributed among all slices for higher cache bandwidth utilization.

C. Cache-aware Mapping

The cache-aware mapping method of CaMDN provides adaptability to varying available cache capacity by generating multiple mapping candidates that target different cache usage levels. As shown in the left part of Figure 6, it takes a hardware configuration, a list of cache usage limitations, and model structures as inputs. For each layer, the layer mapper generates a mapping candidate within each usage limitation. All candidates of the layer are written into a mapping candidate table (MCT) and all MCTs of the model are stored in a model mapping file as the output of the mapping phase.

1) Heuristic-solver-hybrid Layer Mapper: To reduce the additional time cost of generating multiple mapping candidates, CaMDN introduces a heuristic-solver-hybrid layer mapper that combines heuristic rules and an available optimization solver to speed up the mapping process while maintaining high performance of results. Specifically, it first shrinks the problem space according to a set of heuristic rules. These rules improve the utilization of cache line, NPU-private storage and compute resource, and reduce the choices of loop permutation [26]. Second, it constructs a set of disjoint problem subspaces, each of which is an integer programming problem that takes minimal DRAM access as the optimization objective. Third, these problems are solved by the solver, and the result with the minimal DRAM access is selected as the mapping candidate for the layer within a certain cache usage limitation.

2) Layer-block Mapping for Data Reuse: In addition to layer-wise mapping (LWM) mentioned above, CaMDN also uses layer-block mapping (LBM) to store intermediate data between layers fully in cache and allocate zero memory space to these data. LBM significantly reduces memory access during multi-tenant DNN execution. To prevent a model from occupying too much cache space for too long, models are segmented into layer blocks and LBM happens only inside each block. For each layer, an additional LBM candidate is generated alongside multiple LWM candidates.

3) Mapping Candidate Table: Mapping candidate tables (MCTs) are the major outputs of the offline mapping phase. An MCT records all mapping candidates and detailed information of a layer in a compact format instead of unrolled NPU instructions. In this way, much less storage is needed for multiple mapping candidates. As shown in the middle of Figure 6, for each candidate, there is a loop table that records loop permutations and factors, and a cache map table that indicates how tensors are mapped in vcaddr space.

Algorithm 1: Predict near-future shared cache usage and select mapping candidate accordingly for a layer.

Input: t_{cur}: current task with its model information;
MCT_{cur}: MCT for current layer of t_{cur}.
Output: M_{cur}: mapping candidate selected for t_{cur};
P_{cur}: shared cache pages required by M_{cur};
T_{ahead}: timeout threshold for waiting.
Data: T_{next}[numTasks]: profiling-based predicted next reallocation time for each task;
P_{next}[numTasks]: predicted pages needed at next reallocation for each task;
P_{alloc}[numTasks]: allocated pages for each task.

```
// Predict future available pages.
1  Func predAvailPages(T_ahead, t_cur) : P_ahead
2      P_ahead = idlePages();
3      for t_i : runningTasks do
4          if t_i ≠ t_cur and T_next[t_i] < T_ahead then
5              P_ahead += P_alloc[t_i] − P_next[t_i];
6      return P_ahead
// Check if LBM is available.
7  if hasEnabledLBM(t_cur) then
8      M_cur = MCT_cur.LBM;
9      return M_cur, M_cur.P_need, ∞;
10 else if isHeadLayerOfBlock(t_cur.layer_cur) then
11     T_ahead = T_cur + t_cur.layerBlock_cur.T_est × 0.2;
12     P_ahead = predAvailPages(T_ahead, t_cur);
13     if MCT_cur.LBM.P_need < P_ahead then
14         M_cur = MCT_cur.LBM;
15         return M_cur, M_cur.P_need, T_ahead;
// Select a LWM candidate from MCT.
16 T_ahead = T_cur + t_cur.layer_cur.T_est × 0.2;
17 P_ahead = predAvailPages(T_ahead, t_cur);
18 M_cur = MCT_cur.LWMs[0];
19 for M_i : MCT_cur.LWMs do
20     if M_cur.P_need < M_i.P_need ≤ P_ahead then
21         M_cur = M_i;
22 return M_cur, M_cur.P_need, T_ahead;
```

D. Dynamic Cache Allocation Algorithm

The dynamic cache allocation algorithm of CaMDN adjusts cache usage among co-located DNNs by predicting near-future cache usage and selecting mapping candidates accordingly.

As shown in the right part of Figure 6, the algorithm is invoked at the beginning of each layer. First, it predicts the cache usage among tasks in the near future, estimates the available capacity for the layer, and selects the mapping candidate that best fits the available capacity by Algorithm 1. Second, it tries to request the cache pages needed. If the pages become available within a threshold time, the CPTs are modified and the layer is executed with the selected mapping. Otherwise, every time a timeout occurs, it updates the candidate to the one that require fewer pages.

Algorithm 1 describes the detailed process of predicting shared cache usage and selecting a mapping candidate for a layer. The algorithm uses function predAvailPages to predict the available pages P_{ahead} in future time T_{ahead} for task t_{cur} (lines 1-6). The function is based on three global variables, T_{next}, P_{next} and P_{alloc}, which are updated at the end of each layer. The algorithm takes the current task t_{cur} and the current layer's MCT MCT_{cur} as inputs, and produces a mapping candidate M_{cur}, a number of required pages P_{cur}, and a timeout threshold T_{ahead} as outputs. Specifically, it first checks whether LBM is already enabled or can be enabled according to predicted near-future cache usage (lines 7-15). If available, LBM is enabled for this layer block. Otherwise, the LWM candidate that best fits the near-future available capacity is selected (lines 16-22). The timeout threshold T_{ahead} is calculated twice for both steps, using a profiling-based estimation of execution latency $t_{cur}.layer_{cur}.T_{est}$.

TABLE I: Benchmark Models for Multi-tenant Execution

Domain	Model	Abbr.	Type	QoS(ms)
Computer Vision	ResNet50 [27]	RS.	Conv	6.7
	MobileNet-v2 [28]	MB.	DwConv	2.8
	EfficientNet-b0 [29]	EF.	DwConv	2.8
	ViT-base-16 [30]	VT.	Trans	40.0
Natural Language Processing	BERT-base [31]	BE.	Trans	40.0
	GNMT [32]	GN.	LSTM	6.7
Audio Processing	Wav2Vec2-base [33]	WV.	Trans	16.7
Point Cloud	PointPillars [34]	PP.	Conv	100.0

TABLE II: Configuration of the NPU-Integrated SoC

Parameter		Value
NPU	PE Array (per core)	32x32
	Scratchpad (per core)	256KB
	Cores	16
Shared Cache	Total Capacity	16MB
	NPU Ways / Total Ways	12 / 16
	Slices	8
DRAM	Total Bandwidth	102.4GB/s
	Channels	4
Frequency		1GHz

IV. Evaluation

A. Experimental Setup

1) Implementation: The architecture of CaMDN is implemented based on Gemmini [35] in Verilog HDL, and synthesized by Synopsys Design Compiler in a 45nm process technology. OpenRAM [36] is used to generate SRAM macros. For performance evaluation, an in-door cycle-accurate simulator built on DRAMsim3 [37] is used, which is capable to simulate multi-tenant DNNs running on multiple NPUs with a sliced shared cache.

2) Benchmark: Table I shows the eight models chosen to compose the multi-tenant benchmark that feature various domains (CV, NLP, audio processing and point cloud) and different model types (Convolution, Depth-wise Convolution, Transformer and LSTM).

3) Baseline: MoCA [8] and AuRORA [13] are chosen as baseline: MoCA dynamically schedules memory bandwidth; AuRORA dynamically schedules NPUs and memory bandwidth. For fairness, MoCA and AuRORA are implemented with the same hardware configuration as CaMDN. Two versions of CaMDN are set for comparison: CaMDN(HW-only) equally allocates cache capacity among NPUs without dynamic cache scheduling; CaMDN(Full) refers to the full system of the architecture-scheduling co-design.

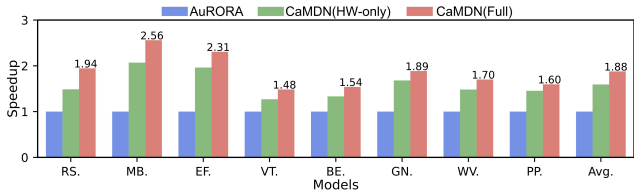

Fig. 7: The model-wise speedup achieved by CaMDN.

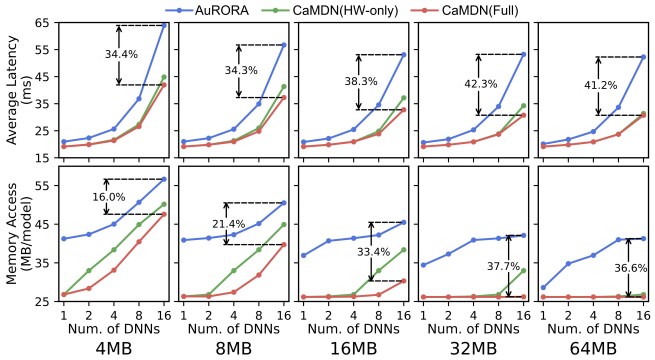

Fig. 8: The average latency and memory access with different scales. The numbers indicate the percentage reduction achieved by CaMDN(Full) compared to the baseline.

4) Experiments: To fully understand the performance improvement, we evaluate CaMDN from three aspects: speedup, scaling and quality-of-service(QoS).

Speedup experiment is conducted with configuration in Table II. we randomly dispatch each model task to one NPU as soon as it finishes its current task. All NPUs are kept busy to maximize the shared cache contention.

Scaling experiment is similar to speedup experiment, but with different cache sizes from 4MB to 64MB, and different numbers of co-located DNNs from 1 to 16.

QoS experiment is conducted with configuration in Table II. CaMDN is integrated with the same memory bandwidth and NPU allocation algorithms as AuRORA. We set tighter latency targets than prior work, as shown in Table I. Following [13], we setup three different QoS levels: QoS-H, QoS-M and QoS-L, respectively referring to 0.8×, 1.0× and 1.2× latency targets. The following metrics are used in comparison: Service Level Agreement (SLA) satisfaction rate indicates the percentage of tasks that meet their deadline; System Throughput (STP) indicates the overall throughput; Fairness indicates the equality degree of the progresses of co-running tasks. Detailed definitions are described in [13].

B. Results Analysis

1) Speedup: Figure 7 shows the model-wise speedup of CaMDN. Since the methods in MoCA and AuRORA are essentially for improving QoS rather than speedup, and they show similar results in this experiment, we choose AuRORA as a representative. CaMDN(Full) achieves an averagely 1.88× speedup, demonstrating that enhancing cache efficiency significantly improves the model speed. CaMDN(Full) surpasses CaMDN(HW-only) by an averagely 1.18× speedup, proving the effectiveness of dynamic cache allocation. CaMDN reaches higher speedup on MobileNet-v2 and EfficientNet-b0 because these models possess larger proportions of intermediate data and CaMDN successfully reduces the memory access by storing intermediate data entirely in cache.

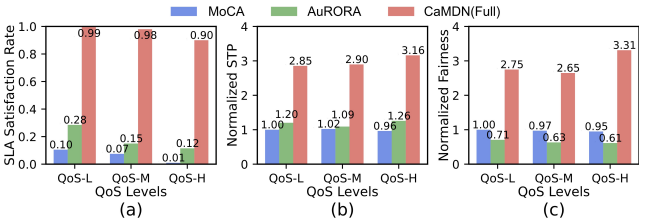

Fig. 9: The QoS improvement achieved by CaMDN.

TABLE III: Area Breakdown of the Architecture of CaMDN

Component	Area(μm^2)	(%)	Component	Area(μm^2)	(%)
NPU	7905k	100.0	Cache Slice	24676k	100.0
Scatchpad	6302k	79.7	Data Array	21878k	88.7
PE Array	1302k	16.5	Tag Array	2398k	9.7
CPT	**73k**	**0.9**	**NEC**	**66k**	**0.3**
others	228k	2.9	others	334k	1.3

2) Scaling: Figure 8 shows the average model latency and memory access with different system scales. Generally, CaMDN(Full) achieves 34.3% to 42.3% latency reductions and 16.0% to 37.7% memory access reductions. Since multi-tenant DNNs are memory-bound, the reduction in memory access contributes a significant portion of performance improvement. As the number of co-located DNNs increases, CaMDN significantly alleviates the performance degradation. As the cache becomes larger, CaMDN generally shows larger enhancement.

3) QoS: As shown in Figure 9, CaMDN achieves averagely 5.9×, 2.5× and 3.0× improvements on SLA satisfaction rate, STP and fairness respectively. This demonstrates that CaMDN can meet more rigorous QoS requirements by improving cache efficiency to reduce the model latency. Notably, CaMDN achieves even higher STP and fairness in QoS-H. The reason is that bandwidth and NPUs are more frequently allocated to slow tasks in QoS-H, while in QoS-L and QoS-M, the improvement of CaMDN largely reduces the need for bandwidth and NPU allocation. Additionally, AuRORA shows lower fairness than MoCA in our experiment, because our QoS targets are much tighter and AuRORA manages to reach higher SLA satisfaction rate at the cost of worsening fairness. Nonetheless, CaMDN can achieve higher performance without sacrificing fairness.

4) Area Overhead: Table III shows the negligible area overhead of an NPU and a cache slice with the configuration in Table II. A CPT contributes 0.9% of the total NPU area. An NEC contributes 0.3% of the total cache slice area.

V. Conclusion

We proposed CaMDN, an architecture-scheduling co-design to enhance cache efficiency for multi-tenant DNNs on integrated NPUs. Compared to prior works that focus on memory bandwidth and/or compute resources, CaMDN is dedicated to solving the inefficiency issue of cache shared by multiple NPUs. CaMDN combines a lightweight architecture, a cache-aware mapping method and a dynamic cache allocation algorithm. CaMDN reduces the memory access by 33.4% and achieves a model speedup of up to 2.56× (1.88× on average). For future work, we believe more efficient scheduling methods could be proposed that take both multi-tenant DNNs and general-purposed programs into consideration.

REFERENCES

[1] G. Williams, "Qualcomm oryon™ cpu," in *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 2024, pp. 1–21.

[2] B. Cohen, M. Subramony, and M. Clark, "Next generation "zen 5" core," in *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 2024, pp. 1–27.

[3] P. Mosur, "Built for the edge: The intel® xeon® 6 soc," in *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 2024, pp. 1–28.

[4] Y. H. Oh, S. Kim, Y. Jin, S. Son, J. Bae, J. Lee, Y. Park, D. U. Kim, T. J. Ham, and J. W. Lee, "Layerweaver: Maximizing resource utilization of neural processing units via layer-wise scheduling," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 584–597.

[5] Y. Choi and M. Rhu, "Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 220–233.

[6] E. Baek, D. Kwon, and J. Kim, "A multi-neural network acceleration architecture," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 940–953.

[7] S.-C. Kao and T. Krishna, "Magma: An optimization framework for mapping multiple dnns on multiple accelerator cores," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 814–830.

[8] S. Kim, H. Genc, V. V. Nikiforov, K. Asanović, B. Nikolić, and Y. S. Shao, "Moca: Memory-centric, adaptive execution for multi-tenant deep neural networks," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 828–841.

[9] S. Ghodrati, B. H. Ahn, J. K. Kim, S. Kinzer, B. R. Yatham, N. Alla, H. Sharma, M. Alian, E. Ebrahimi, N. S. Kim *et al.*, "Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 681–697.

[10] S. Kim, H. Kwon, J. Song, J. Jo, Y.-H. Chen, L. Lai, and V. Chandra, "Dream: A dynamic scheduler for dynamic real-time multi-model ml workloads," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 4*, 2023, pp. 73–86.

[11] F. G. Blanco, E. Russo, M. Palesi, D. Patti, G. Ascia, and V. Catania, "A deep reinforcement learning based online scheduling policy for deep neural network multi-tenant multi-accelerator systems," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.

[12] H. Kwon, L. Lai, M. Pellauer, T. Krishna, Y.-H. Chen, and V. Chandra, "Heterogeneous dataflow accelerators for multi-dnn workloads," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 71–83.

[13] S. Kim, J. Zhao, K. Asanovic, B. Nikolic, and Y. S. Shao, "Aurora: Virtualized accelerator orchestration for multi-tenant workloads," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 62–76.

[14] Z. Liu, J. Leng, Z. Zhang, Q. Chen, C. Li, and M. Guo, "Veltair: towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022, pp. 388–401.

[15] M. K. Qureshi and Y. N. Patt, "Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2006, pp. 423–432.

[16] D. Sanchez and C. Kozyrakis, "Vantage: Scalable and efficient fine-grain cache partitioning," in *Proceedings of the 38th annual international symposium on Computer architecture*, 2011, pp. 57–68.

[17] A. Herdrich, E. Verplanke, P. Autee, R. Illikkal, C. Gianos, R. Singhal, and R. Iyer, "Cache qos: From concept to reality in the intel® xeon® processor e5-2600 v3 product family," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2016, pp. 657–668.

[18] J. Park, S. Park, and W. Baek, "Copart: Coordinated partitioning of last-level cache and memory bandwidth for fairness-aware workload consolidation on commodity servers," in *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019, pp. 1–16.

[19] C. F. Fajardo, Z. Fang, R. Iyer, G. F. Garcia, S. E. Lee, and L. Zhao, "Buffer-integrated-cache: A cost-effective sram architecture for handheld and embedded platforms," in *Proceedings of the 48th Design Automation Conference (DAC)*, 2011, pp. 966–971.

[20] J. Cong, M. A. Ghodrat, M. Gill, C. Liu, and G. Reinman, "Bin: A buffer-in-nuca scheme for accelerator-rich cmps," in *Proceedings of the 2012 ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, 2012, pp. 225–230.

[21] J. Zuckerman, D. Giri, J. Kwon, P. Mantovani, and L. P. Carloni, "Cohmeleon: Learning-based orchestration of accelerator coherence in heterogeneous socs," in *54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 350–365.

[22] A. Karatzas and I. Anagnostopoulos, "Omniboost: Boosting throughput of heterogeneous embedded devices under multi-dnn workload," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[23] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma *et al.*, "Ten lessons from three generations shaped google's tpuv4i: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1–14.

[24] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2019, pp. 304–315.

[25] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 554–566.

[26] S. Dave, Y. Kim, S. Avancha, K. Lee, and A. Shrivastava, "Dmazerunner: Executing perfectly nested loops on dataflow accelerators," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 1–27, 2019.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[28] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[29] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*. PMLR, 2019, pp. 6105–6114.

[30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *9th International Conference on Learning Representations, ICLR 2021*. OpenReview.net, 2021.

[31] J. D. M.-W. C. Kenton and L. K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of naacL-HLT*, vol. 1. Minneapolis, Minnesota, 2019, p. 2.

[32] Y. Wu, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.

[33] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 12 449–12 460, 2020.

[34] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12 697–12 705.

[35] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 769–774.

[36] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "Openram: An open-source memory compiler," in *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2016, pp. 1–6.

[37] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.